

---

# Práctica 11: trazador de rayos

---

**Fecha de entrega:** 23 de abril de 2026, 13:50

## Material proporcionado:

| Fichero            | Explicación                                                      |
|--------------------|------------------------------------------------------------------|
| TrazadorBasico.zip | Versión funcional del trazador básico creado en clase de teoría. |

**Objetivo:** Implementación de la arquitectura básica de un trazador de rayos

## 1. Instrucciones

La práctica consiste en la creación de un pequeño trazador de rayos capaz de renderizar unas pocas esferas de colores.

Para su construcción se parte de un trazador de rayos minimalista y poco estructurado que genera una esfera roja en el centro de la pantalla. El código proporcionado puede utilizarse tal cual y extenderlo o puede utilizarse a modo de inspiración para construir uno similar pero con estructura de ficheros, nombres de clases y métodos diferentes. Eso sí, no deberías hacer cambios drásticos en la organización de clases y métodos.

Las extensiones se van describiendo poco a poco más abajo. Aunque se pasará por distintas “versiones” del trazador, sólo será necesario incluir la versión final del código. La entrega debe incluir esos ficheros de tal forma que baste compilar todos los `.cpp` desde el directorio raíz de código (en Linux debería bastar con algo como `g++ *.cpp`).

## 2. Punto de partida

El material proporcionado consiste en una serie de ficheros de C++.

Crea un proyecto para compilarlo utilizando tu IDE favorito. Al ejecutarlo debería generarse una imagen como la de la figura 1.

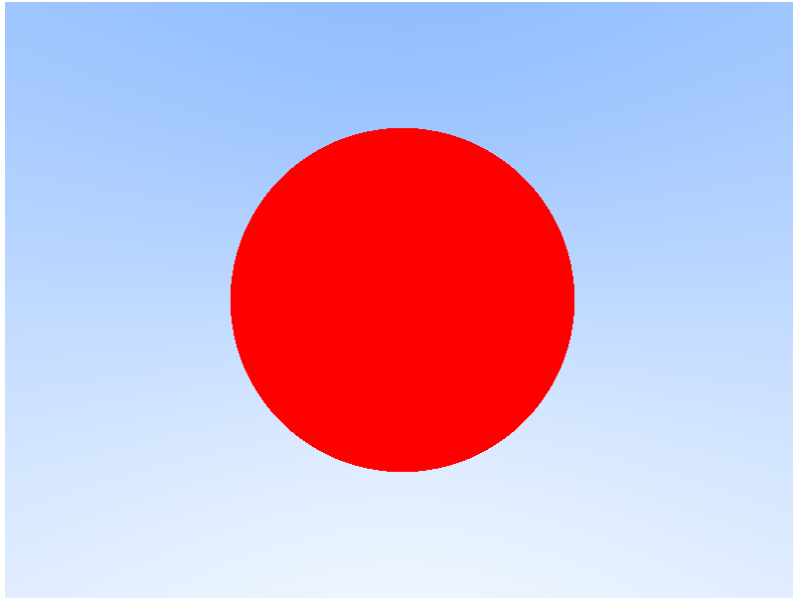


Figura 1: Trazador de rayos de partida

### 3. Clase **Shape**

Para abstraer el tipo de forma/figura/objeto que maneja el trazador, implementa una clase (abstracta) que represente uno de estos objetos. Su nombre puede ser, por ejemplo, `Shape`. Debe tener un método como:

```
virtual bool Intersect(const Ray &ray, float tMin, float tMax) const = 0;
```

que determina si el rayo pasado como parámetro interseca con el objeto. La colisión se evalúa únicamente en el segmento del rayo en el que el multiplicador de su dirección ( $t$ , parámetro de la función `at`) está entre `tMin` y `tMax`.

Posteriormente extenderemos la clase con un método adicional.

### 4. Clase **Sphere**

Implementa la clase `Sphere` que represente una esfera. En su constructor recibe el centro y radio.

Modifica el `main` del trazador para que en lugar de tener los puntos cableados de la esfera roja inicial, utilice esta clase.

### 5. Clase **Renderer**

Implementa la clase `Renderer` que se encargue de la generación de la imagen final. En su constructor debe recibir el `Film` y `Camera` que debe utilizar, así como un objeto `Shape` a renderizar.

Tendrá dos métodos:

- `void Render();` que genera la escena.
- `Color ray_color(const Ray& r);` que devuelve el color del rayo lanzado sobre la geometría.

Las implementaciones de ambos métodos pueden extraerse del código de la función `main` del trazador de partida.

Tras la inclusión de la clase, la siguiente función `main` debería conseguir el mismo resultado que el trazador original:

```
int main() {

    ofstream outFile("imagen.ppm");

    Film film(800, 600, outFile);
    Camera camera(film,
                  glm::vec3(0, 0, 0),
                  glm::vec3(0, 0, -1),
                  glm::vec3(0, 1, 0));

    Sphere obj(glm::vec3(0,0,-1), 0.5);

    Renderer renderer(film, camera, obj);

    renderer.Render();

    return 0;
}
```

## 6. Clase Material

El siguiente paso es añadir un primitivo soporte para materiales. Implementa una clase `Material` que, de momento, almacene el color base del material y tenga un método para acceder a él.

Extiende entonces la clase `Sphere` de forma que almacene el material del que está hecha. Como en el futuro muchos objetos podrán utilizar el mismo material, es mejor no almacenar una copia del material, sino que se pueda compartir entre varios objetos. Para eso es aconsejable utilizar algún mecanismo cómodo para la gestión de los punteros y su destrucción, como `shared_ptr` de la librería de C++.

Una posible función `main` sería:

```
#include <memory>
// ...

int main() {

    ofstream outFile("imagen.ppm");

    Film film(800, 600, outFile);
    Camera camera(film,
                  glm::vec3(0, 0, 0),
                  glm::vec3(0, 0, -1),
                  glm::vec3(0, 1, 0));

    std::shared_ptr<Material> rojo = std::make_shared<Material>(RED);
    Sphere obj(glm::vec3(0,0,-1), 0.5, rojo);

    Renderer renderer(film, camera, obj);
```

```

    renderer.Render();

    return 0;
}

```

## 7. Clase ShapeIntersection

Para que el `Renderer` pueda aprovechar el material del que está hecho el objeto, necesita una versión mejorada del método que calcula la intersección (o *hit*) de un rayo con el objeto. Crea una *nueva* versión del método `Intersect` que reciba un parámetro adicional *de salida* con la información de la colisión.

La clase puede llamarse `ShapeIntersection` y, de momento, basta con que tenga el material del que está hecho el objeto con el que se ha colisionado el rayo<sup>1</sup>.

Fíjate que mantendremos las dos implementaciones: una que simplemente devuelva si hubo colisión o no y otra más completa pero más lenta que además devuelve la información sobre la colisión. De esta forma podremos utilizar la versión simple para los *shadow rays*.

Utiliza en el `Renderer` la versión extendida para utilizar el color del material contra el que ha chocado el rayo a la hora de establecer el color del pixel.

## 8. Clase Scene

Implementa una clase `Scene` que herede de `Shape` y almacene una lista de formas para que el `Renderer` pueda dibujar más de un objeto.

El siguiente código construye la “escena” mostrada en la figura 2:

```

int main() {

    // ...

    shared_ptr<Material> azul = make_shared<Material>(BLUE);
    shared_ptr<Material> amarillo = make_shared<Material>(YELLOW);
    shared_ptr<Material> rojo = make_shared<Material>(RED);

    shared_ptr<Sphere> obj3 =
        make_shared<Sphere>(glm::vec3(-1,0,-1), 0.5, azul);
    shared_ptr<Sphere> obj2 =
        make_shared<Sphere>(glm::vec3(0,0,-2), 1.0, amarillo);
    shared_ptr<Sphere> obj1 =
        make_shared<Sphere>(glm::vec3(1,0,-1), 0.5, rojo);

    Scene scene;
    scene.Add(obj1); scene.Add(obj2); scene.Add(obj3);

    Renderer renderer(film, camera, scene);

    // ...
}

```

Tendrás que implementar correctamente sus métodos `Intersect`. Ten en cuenta que al haber varios objetos en la escena, el rayo podría colisionar con más de uno. En ese

---

<sup>1</sup>Recuerda que es mejor no devolver una copia del material.

caso el método debe devolver la información de *la primera colisión*. Para eso necesitarás extender la clase `ShapeIntersection` para devolver no solo el material sino también el punto en el rayo en el que se ha producido la colisión.

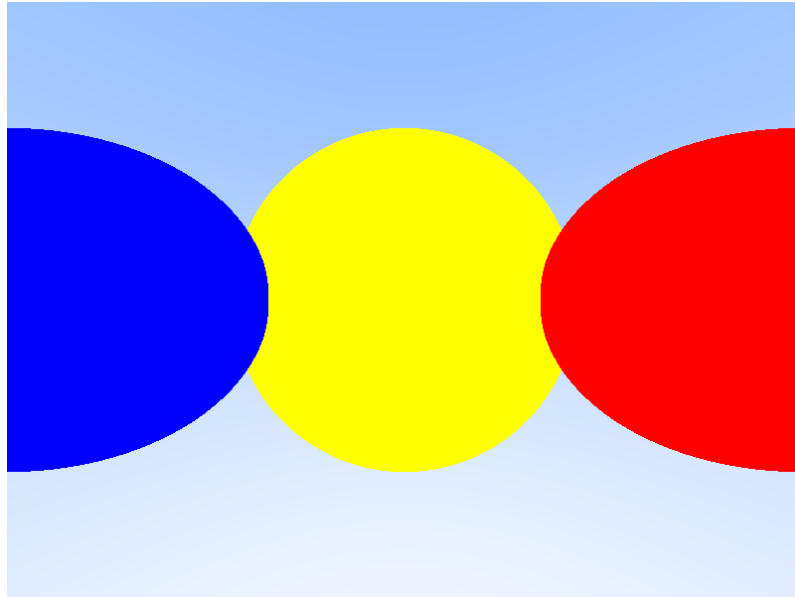


Figura 2: Tres esferas

Utilizando una escena similar a la anterior, crea una imagen en FullHD (1920×1080) e incorpórala a la entrega *tras convertirla a formato PNG*<sup>2</sup>.

## 9. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- Un directorio `Src` con el código fuente completo del trazador de rayos. El main debe construir una escena similar a la del apartado 8. El código debe compilar tal y como se ha descrito en la sección 1.
- Una imagen con el resultado de dibujar la escena con el trazador de rayos implementado.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.

---

<sup>2</sup>La imagen en formato PPM en FullHD ocupa más de 20Mb, mientras que en PNG no llega a los 50K.